

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ ДЕРЖАВНИЙ ВИЩИЙ
НАВЧАЛЬНИЙ ЗАКЛАД “УЖГОРОДСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ” ІНЖЕНЕРНО-ТЕХНІЧНИЙ ФАКУЛЬТЕТ КАФЕДРА
КОМП’ЮТЕРНИХ СИСТЕМ ТА МЕРЕЖ

ЗВІТ

до лабораторної роботи №5

з дисципліни “Програмування мікроконтролерів”

Студента 2-го курсу

Спеціальності 123- “Комп’ютерна інженерія”

Скоробогатого Максима Максимовича

м. Ужгород – 2026 р.

ЛАБОРАТОРНА РОБОТА №5

Динамічна індикація, таймери, робота з логічним аналізатором у інтегрованому середовищі PSoC® Creator™

Мета: Ознайомитися з принципами роботи динамічної індикації та налаштуванні переривань у мікроконтролерних системах на базі PSoC. Набути практичних навичок створення проєктів у інтегрованому середовищі PSoC® Creator™, налаштування апаратних компонентів для керування індикаторами через зсувні регістри та реалізації ефективного виведення даних. Також набути практичних навичок використання логічного аналізатора на прикладі SaleaeLogic.

ПОРЯДОК ВИКОНАННЯ РОБОТИ

1. Копіюємо проєкт Лабораторної роботи №4.

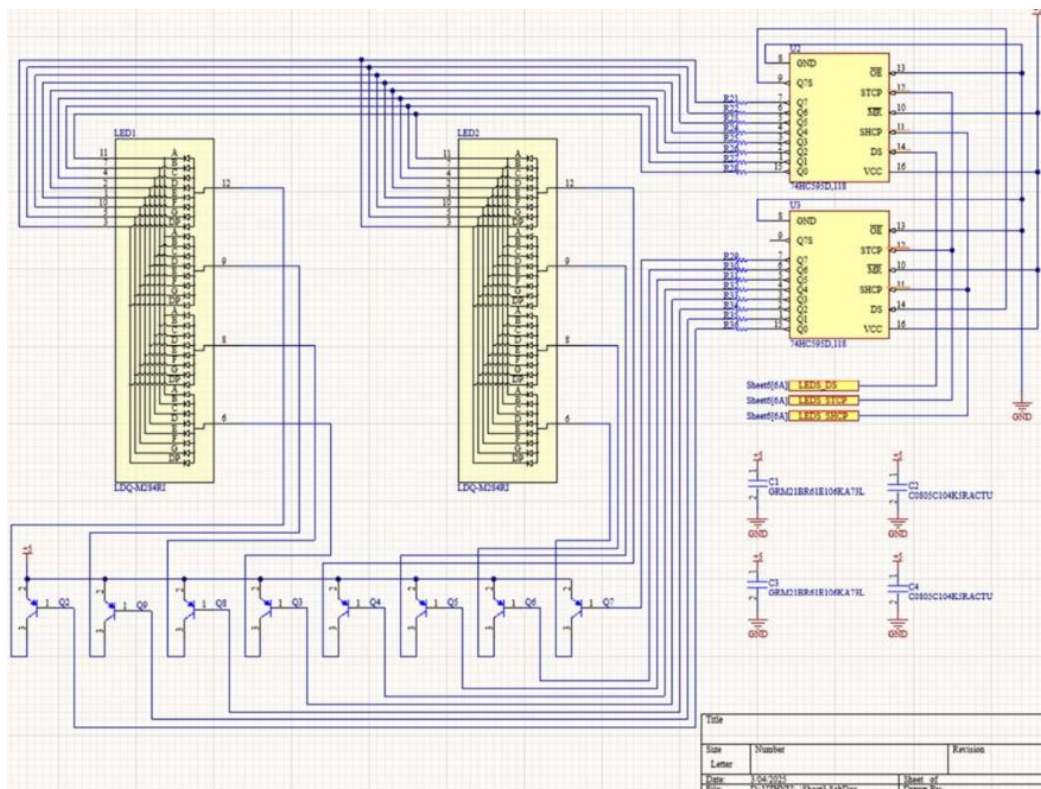


Рисунок 1 – Схема підключення семисегментних індикаторів до регістрів зсуву

2. На рис. 1 зображено схему підключення семисегментних індикаторів до зсувних регістрів. Як видно на схемі, всі семисегментні символи паралельно підключені до першого зсувного регістра, а всі аноди до іншого. Тобто перші 8 біт керують символом який виводиться, а наступні 8 біт керують номером символа який потрібно увімкнути.

3. Пишемо код в `main()`, який відправляє дані по чергово для кожного символа, на індикаторах має бути послідовність значено від 7 до 0. Приклад показано на рис. 2.

```

72 int main(void) {
73     CyGlobalIntEnable;
74
75     // Масив з числами від 7 до 0 для перевірки динамічної індикації
76     uint8_t data[8] = {7, 6, 5, 4, 3, 2, 1, 0};
77
78     for(;;) {
79         // Проходимося по всіх 8 індикаторах
80         for (int i = 0; i < 8; i++) {
81             FourDigit74HC595_sendOneDigit(i, data[i], 0);
82         }
83     }
84 }

```

Рисунок 2 – Функція реалізації динамічної індикації

4. Переходимо на вкладку **TopDesignn.cysch**. Редагуємо проєкт видаливши компоненти Лабораторної роботи №1 та №2, а саме Button, LED_R, LED_B, LED_G та SW_Tx_UART. З каталогу **Cypress** додаємо на схему таймер **Timer [v2.80]** (підкаталог Digital, папка Functions) та з підкаталогу System, компоненту **Interrupt [v1.71]** (рис. 3).

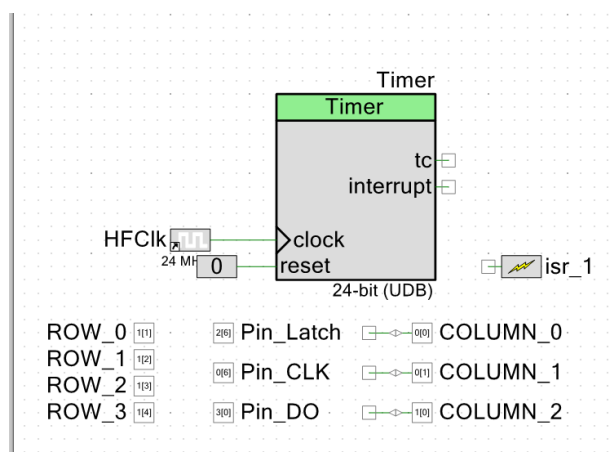


Рисунок 3 – Компоненти Timer [v2.80] та Interrupt [v1.71]

Виконуємо налаштування таймеру як зображено на рис 4, а саме:

– Name: Timer;

– Configure:

- Resolution: 24 bit;
- Implementation: UDB;
- Period: 24000 (1 ms);
- Trigger Mode: None;
- Capture Mode: None;
- Enable Mode: Software Only;
- Run Mode: Continuous;
- Interrupts: —.

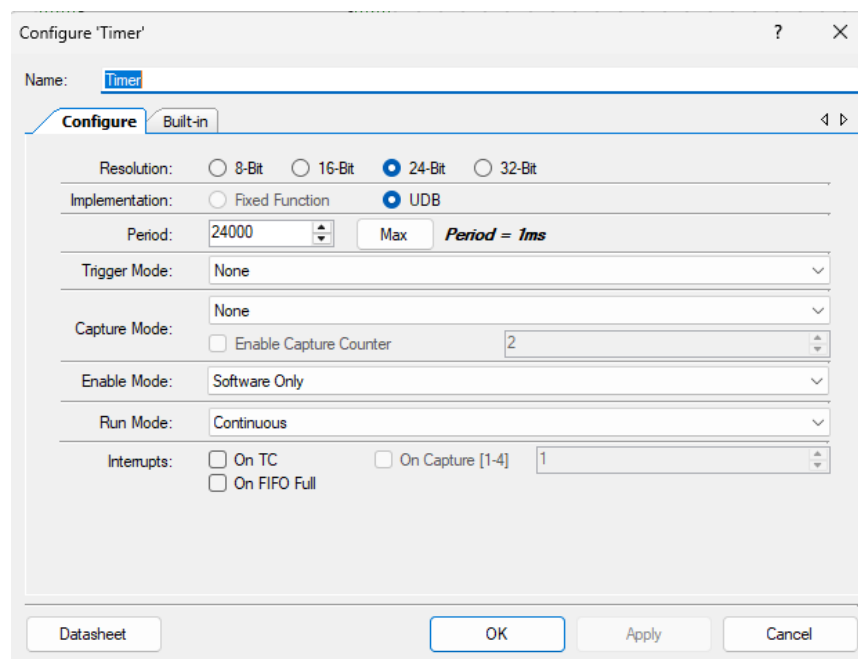


Рисунок 4 – Налаштування компоненти Timer [v2.80]

Компоненту Interrupt [v1.71] лише переіменовуємо Timer_Int. Кінцевий варіант проєкту, після налаштувань, зображено на рис. 5.

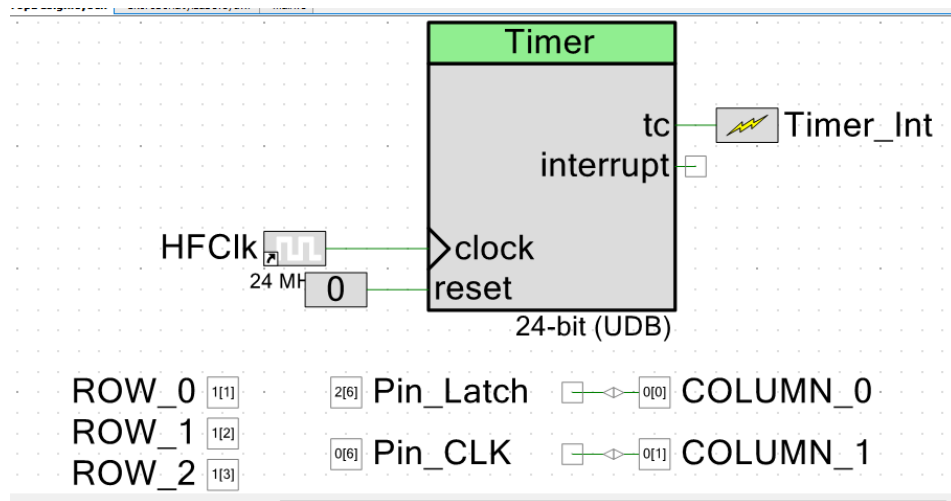


Рисунок 5 – Проєкт Лабораторної роботи №5

5. Пишемо функцію для обробника переривань (рис. 6).

```
//Обробник переривання таймера
CY_ISR(Timer_Int_Handler2)
{
    uint8_t data[8] = {7, 6, 5, 4, 3, 2, 1, 0};
    FourDigit74HC595_sendOneDigit(led_counter, data[led_counter], 0);
    led_counter++;
    if(led_counter > 7) led_counter = 0;
}
```

Рисунок 6 – Обробник переривань

В main() запускаємо таймер та переривання (рис. 7).

```
82 int main(void)
83 {
84     CyGlobalIntEnable;
85     // Запускаємо таймер
86     Timer_Start();
87     // Запускаємо обробник переривання
88     Timer_Int_StartEx(Timer_Int_Handler2);
89     for(;;)
90     {
91     }
92 }
93
94
95
```

Рисунок 7 – Запуск таймера та переривання

6. Проводимо експеримент з періодом таймера. Змінюємо його значення на 10 мс, та на 100 мс:

Аналіз результатів

- **Базовий період (1 мс):**

Кожен індикатор горить 1 мілісекунду. Оскільки в нас 8 індикаторів, повний цикл відмальовування всього дисплея займає 8 мс. Частота оновлення екрана становить $1000 / 8 = 125$ Гц. Оскільки наше око не сприймає мерехтіння на частотах понад 50-60 Гц, мозок "зливає" ці спалахи разом, і ми бачимо ілюзію постійного горіння.

- **Експеримент 1 (10 мс):**

Повний цикл відмальовування тепер займає 80 мс. Частота оновлення падає до $1000 / 80 = 12.5$ Гц. При такій частоті ілюзія починає руйнуватися, і ми бачимо неприємне мерехтіння дисплея.

- **Експеримент 2 (100 мс):**

Повний цикл займає 800 мс. Частота оновлення становить 1.25 Гц. Ілюзія цілісного зображення зникає повністю. Можна чітко побачити повний алгоритм роботи: цифри запалюватимуться по черзі (одна за одною) з кроком в 0.1 секунди, ніби біжучий рядок

7. Відкриваємо програму Logic. Підключаємо логічний аналізатор до плати та до комп'ютера.

8. Відкриваємо програму Logic та налаштовуємо декодування SPI (рис. 8).

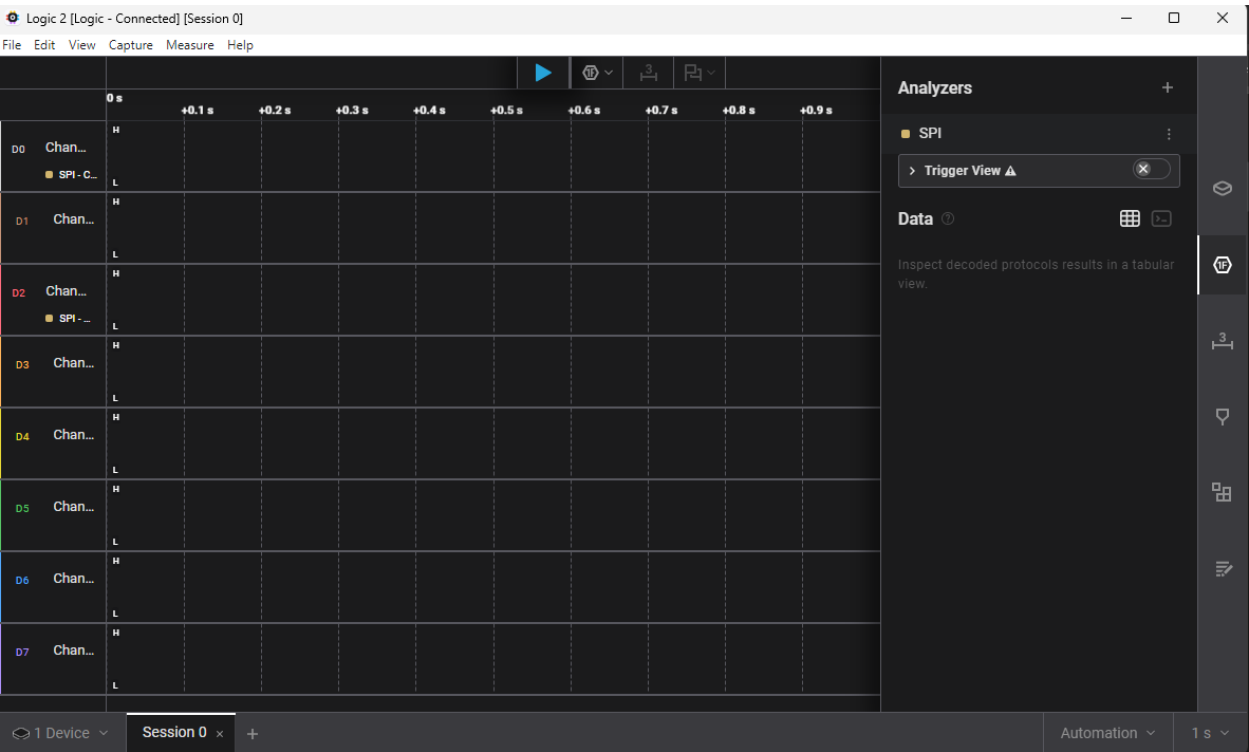


Рисунок 8 – Logic. Налаштування декодування SPI

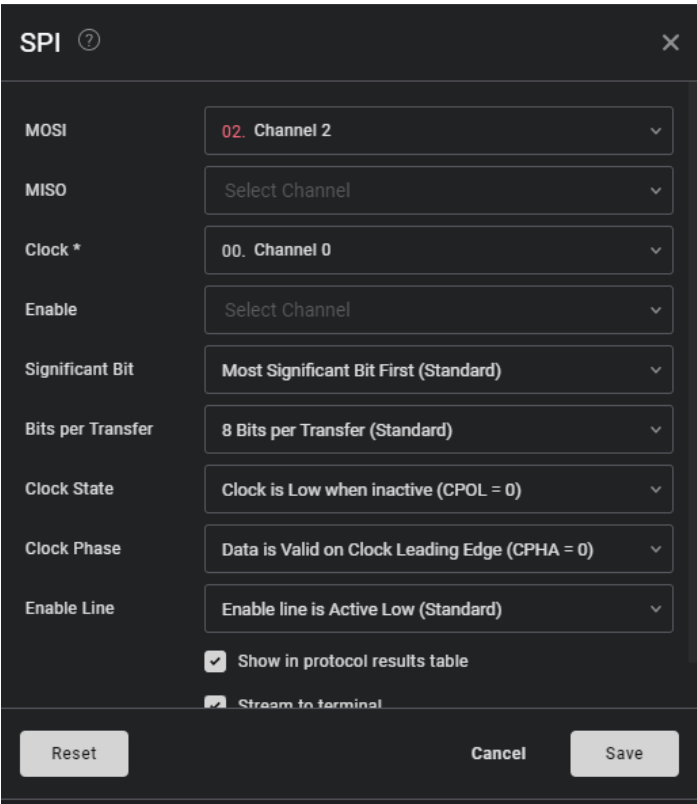


Рисунок 9 – Logic. Вибір каналу

Додамо ще одне декодування SPI, з тими ж налаштуваннями. Правою кнопкою миші виконаємо налаштування виводу у Hexadecimal та Binary формати (рис.10).

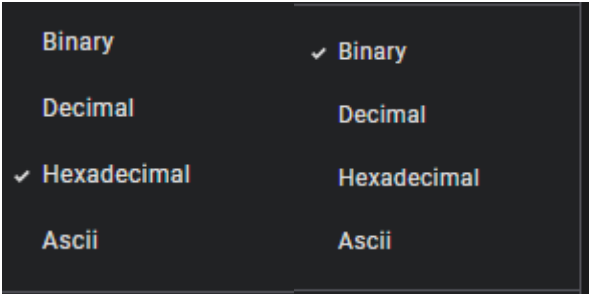


Рисунок 10 – Logic. Вибір формату відображення

Тепер в головному вікні програми Logic можна дивитись як саме передаються дані на регістр зсуву (рис. 11).

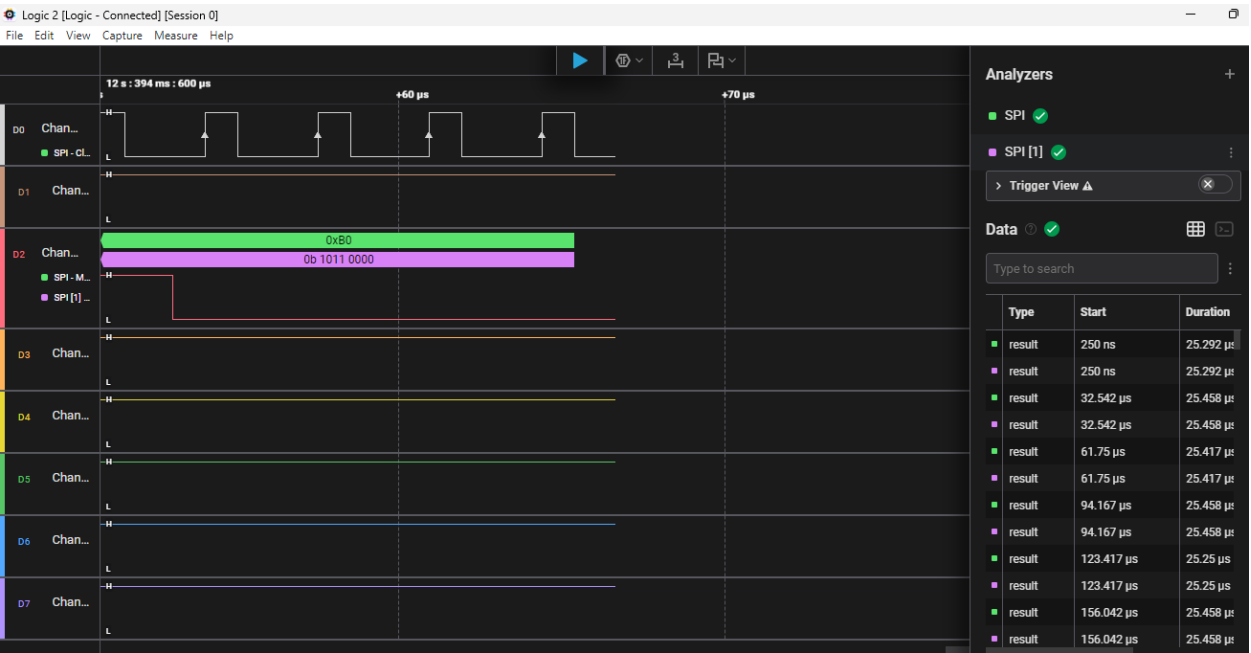


Рисунок 11 – Передача даних на регістр зсуву

9. Завдання для самостійного виконання:

Завдання1. Вивести на дисплей число та декодувати його за допомогою логічного аналізатора.

Формула по якій визначається число:

НОМЕР по списку = 10.

Число яке потрібно вивести та декодувати це $10 + 26 = 36$.

Лістинг коду

```
#include "project.h"

static uint8_t LED_NUM[] = {
    0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8,
    0x80, 0x90, 0xBF, 0x88, 0x83, 0xC6, 0xA1, 0x86,
    0x8E, 0x7F
};

volatile uint8_t display_data[8] = {6, 3, 20, 20, 20, 20, 20, 20};

static void FourDigit74HC595_sendData(uint8_t data) {
    for (uint8_t i = 0; i < 8; i++) {
        if (data & (0x80 >> i)) Pin_DO_Write(1);
        else Pin_DO_Write(0);

        Pin_CLK_Write(1);
        Pin_CLK_Write(0);
    }
}

static void FourDigit74HC595_sendOneDigit(uint8_t position, uint8_t digit,
uint8_t dot) {
    if (position >= 8) {
        FourDigit74HC595_sendData(0xFF);
        FourDigit74HC595_sendData(0xFF);
        return;
    }

    FourDigit74HC595_sendData(0xFF & ~(1 << position));

    if (digit == 20) {
        FourDigit74HC595_sendData(0xFF);
    } else if (dot) {
        FourDigit74HC595_sendData(LED_NUM[digit] & 0x7F);
    } else {
        FourDigit74HC595_sendData(LED_NUM[digit]);
    }

    Pin_Latch_Write(1);
    Pin_Latch_Write(0);
}

static uint8_t led_counter = 0;
```

```

CY_ISR(Timer_Int_Handler2)
{
    FourDigit74HC595_sendOneDigit(led_counter, display_data[led_counter], 0);
    led_counter++;
    if (led_counter > 7) led_counter = 0;
}
int main(void)
{
    CyGlobalIntEnable;
    Timer_Start();
    Timer_Int_StartEx(Timer_Int_Handler2);

    for(;;)
    {
    }
}

```

На рис. 12 у програмі Logic показано процес передачі даних.

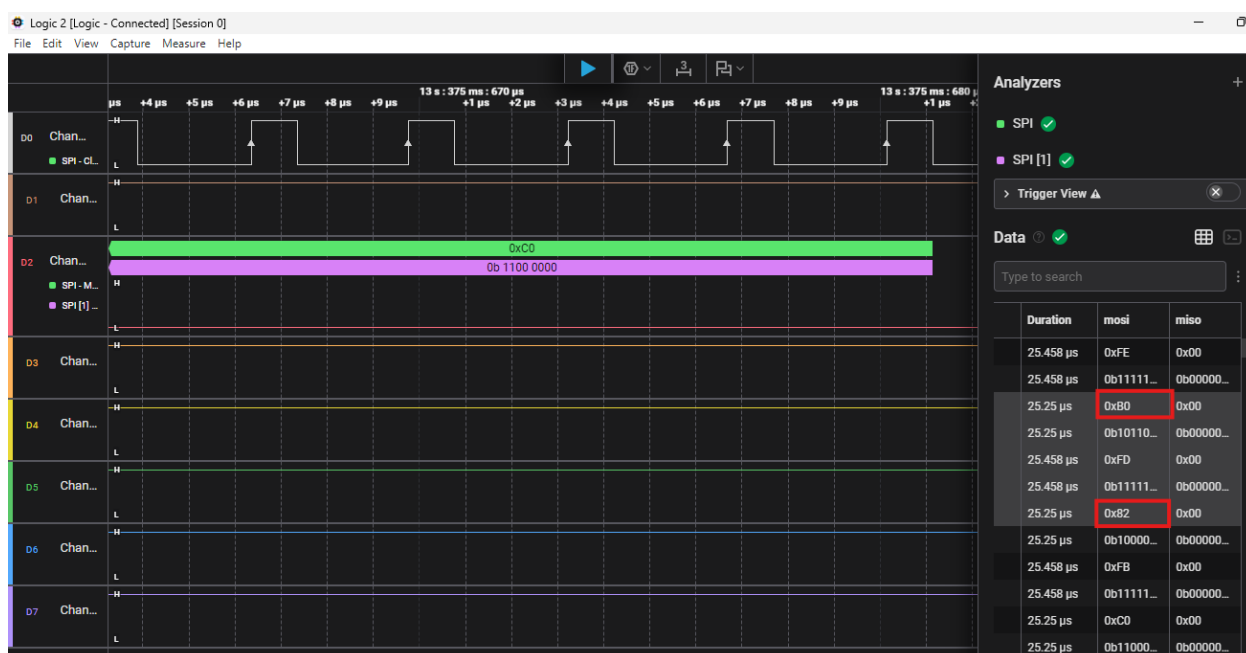


Рисунок 12 – процес передачі чисел 3 і 6

Завдання 2. Додати можливість вводу паролю на семисегментний дисплей, після натиску на кнопку «#». Перевірити введений пароль з тим, який Ви оголосите самостійно в коді, якщо пароль вірний, виконати якусь дію на семисегментних індикаторах. Також якщо пароль не вірний, потрібно сповістити користувача іншим паттерном мигання на семисегментних індикаторах.

Запуск: Натискаємо #. Програма перемикається в режим пароля, а на екрані з'являються тире ----.

Ввід: Натискаємо цифри. Сканування клавіатури фіксує їх, записує в пам'ять і по черзі виводить на дисплей (наприклад, 1---, потім 12--).

Аналіз: Коли введено рівно 4 цифри, мікроконтролер порівнює ввід із кодом 1234.

Вердикт:

Вірно: Дисплей тричі мигає вісімками (8888).

Помилка: Дисплей тричі мигає прочерками (----).

Скидання: Після мигання система автоматично повертається до початкового стану і знову показує число 36.

Лістинг коду

```
#include "project.h"

static uint8_t LED_NUM[] = {
    0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8,
    0x80, 0x90, 0xBF, 0x88, 0x83, 0xC6, 0xA1, 0x86,
    0x8E, 0x7F
};

volatile uint8_t display_data[8] = {6, 3, 20, 20, 20, 20, 20, 20};

static void FourDigit74HC595_sendData(uint8_t data) {
    for (uint8_t i = 0; i < 8; i++) {
        if (data & (0x80 >> i)) Pin_DO_Write(1);
        else Pin_DO_Write(0);

        Pin_CLK_Write(1);
        Pin_CLK_Write(0);
    }
}

static void FourDigit74HC595_sendOneDigit(uint8_t position, uint8_t digit,
uint8_t dot) {
    if (position >= 8) {
        FourDigit74HC595_sendData(0xFF);
        FourDigit74HC595_sendData(0xFF);
        return;
    }

    FourDigit74HC595_sendData(0xFF & ~(1 << position));

    if (digit == 20) {
        FourDigit74HC595_sendData(0xFF);
    } else if (dot) {
        FourDigit74HC595_sendData(LED_NUM[digit] & 0x7F);
    } else {
```

```

        FourDigit74HC595_sendData(LED_NUM[digit]);
    }

    Pin_Latch_Write(1);
    Pin_Latch_Write(0);
}

const uint8_t keyMap[4][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9},
    {10, 0, 11}
};

uint8_t ScanMatrixKeypad(void) {
    uint8_t result = 0xFF;

    for (uint8_t row = 0; row < 4; row++) {
        ROW_0_Write(1); ROW_1_Write(1); ROW_2_Write(1); ROW_3_Write(1);

        if(row == 0) ROW_0_Write(0);
        if(row == 1) ROW_1_Write(0);
        if(row == 2) ROW_2_Write(0);
        if(row == 3) ROW_3_Write(0);

        CyDelayUs(50);

        if (COLUMN_0_Read() == 0) { result = keyMap[row][0]; break; }
        if (COLUMN_1_Read() == 0) { result = keyMap[row][1]; break; }
        if (COLUMN_2_Read() == 0) { result = keyMap[row][2]; break; }
    }

    ROW_0_Write(1); ROW_1_Write(1); ROW_2_Write(1); ROW_3_Write(1);
    return result;
}

static uint8_t led_counter = 0;

CY_ISR(Timer_Int_Handler2)
{
    FourDigit74HC595_sendOneDigit(led_counter, display_data[led_counter], 0);
    led_counter++;
    if (led_counter > 7) led_counter = 0;
}

int main(void)
{
    CyGlobalIntEnable;
    Timer_Start();
    Timer_Int_StartEx(Timer_Int_Handler2);

    uint8_t password[4] = {1, 2, 3, 4};
    uint8_t input_buffer[4] = {0};
    uint8_t input_index = 0;
    uint8_t is_entering_password = 0;
    uint8_t last_key = 0xFF;

    for(;;)
    {
        uint8_t key = ScanMatrixKeypad();

        if (key != 0xFF && key != last_key)
        {
            if (key == 11)

```

```

        {
            is_entering_password = 1;
            input_index = 0;
            display_data[3] = 10; display_data[2] = 10; display_data[1] =
10; display_data[0] = 10;
            display_data[7] = 20; display_data[6] = 20; display_data[5] =
20; display_data[4] = 20;
        }
        else if (is_entering_password == 1 && key < 10)
        {
            input_buffer[input_index] = key;
            display_data[3 - input_index] = key;
            input_index++;

            if (input_index == 4)
            {
                if (input_buffer[0] == password[0] && input_buffer[1] ==
password[1] &&
                    input_buffer[2] == password[2] && input_buffer[3] ==
password[3])
                {
                    for(int b = 0; b < 3; b++) {
                        display_data[3] = 8; display_data[2] = 8;
display_data[1] = 8; display_data[0] = 8;
                        CyDelay(300);
                        display_data[3] = 20; display_data[2] = 20;
display_data[1] = 20; display_data[0] = 20;
                        CyDelay(300);
                    }
                }
                else
                {
                    for(int b = 0; b < 3; b++) {
                        display_data[3] = 10; display_data[2] = 10;
display_data[1] = 10; display_data[0] = 10;
                        CyDelay(300);
                        display_data[3] = 20; display_data[2] = 20;
display_data[1] = 20; display_data[0] = 20;
                        CyDelay(300);
                    }
                }

                is_entering_password = 0;
                display_data[0] = 6; display_data[1] = 3;
                display_data[2] = 20; display_data[3] = 20;
            }
        }
    }

    last_key = key;
    CyDelay(10);
}
}

```

Завдання 3. Виконати циклічний вивід певного набору символів на семисегментних індикаторах за варіантами:

Варіант	Зсув
10	Кожен третій на 3 позиції вліво

Лістинг коду

```
#include "project.h"

static uint8_t LED_NUM[] = {
    0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8,
    0x80, 0x90, 0xBF, 0x88, 0x83, 0xC6, 0xA1, 0x86,
    0x8E, 0x7F
};

volatile uint8_t display_data[8] = {1, 2, 3, 4, 5, 6, 7, 8};

static void FourDigit74HC595_sendData(uint8_t data) {
    for (uint8_t i = 0; i < 8; i++) {
        if (data & (0x80 >> i)) Pin_DO_Write(1);
        else Pin_DO_Write(0);

        Pin_CLK_Write(1);
        Pin_CLK_Write(0);
    }
}

static void FourDigit74HC595_sendOneDigit(uint8_t position, uint8_t digit,
uint8_t dot) {
    if (position >= 8) {
        FourDigit74HC595_sendData(0xFF);
        FourDigit74HC595_sendData(0xFF);
        return;
    }

    FourDigit74HC595_sendData(0xFF & ~(1 << position));

    if (digit == 20) {
        FourDigit74HC595_sendData(0xFF);
    } else if (dot) {
        FourDigit74HC595_sendData(LED_NUM[digit] & 0x7F);
    } else {
        FourDigit74HC595_sendData(LED_NUM[digit]);
    }

    Pin_Latch_Write(1);
    Pin_Latch_Write(0);
}

static uint8_t led_counter = 0;

CY_ISR(Timer_Int_Handler2)
{
    FourDigit74HC595_sendOneDigit(led_counter, display_data[led_counter], 0);
    led_counter++;
    if (led_counter > 7) led_counter = 0;
}
```

```

}

int main(void)
{
    CyGlobalIntEnable;
    Timer_Start();
    Timer_Int_StartEx(Timer_Int_Handler2);

    uint8_t temp[8];

    for(;;)
    {
        CyDelay(1000);

        for(int i = 0; i < 8; i++) {
            temp[i] = display_data[i];
        }

        for(int i = 0; i < 8; i++) {
            display_data[(i + 3) % 8] = temp[i];
        }
    }
}

```

Завдання 4. На перших чотирьох індикаторах реалізувати відлік 5 хвилин.

Лістинг коду

```

#include "project.h"

static uint8_t LED_NUM[] = {
    0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8,
    0x80, 0x90, 0xBF, 0x88, 0x83, 0xC6, 0xA1, 0x86,
    0x8E, 0x7F
};

volatile uint8_t display_data[8] = {20, 20, 20, 20, 20, 20, 20, 20};

static void FourDigit74HC595_sendData(uint8_t data) {
    for (uint8_t i = 0; i < 8; i++) {
        if (data & (0x80 >> i)) Pin_DO_Write(1);
        else Pin_DO_Write(0);

        Pin_CLK_Write(1);
        Pin_CLK_Write(0);
    }
}

static void FourDigit74HC595_sendOneDigit(uint8_t position, uint8_t digit,
uint8_t dot) {
    if (position >= 8) {
        FourDigit74HC595_sendData(0xFF);
        FourDigit74HC595_sendData(0xFF);
        return;
    }

    FourDigit74HC595_sendData(0xFF & ~(1 << position));

    if (digit == 20) {
        FourDigit74HC595_sendData(0xFF);
    }
}

```

```

    } else if (dot) {
        FourDigit74HC595_sendData(LED_NUM[digit] & 0x7F);
    } else {
        FourDigit74HC595_sendData(LED_NUM[digit]);
    }

    Pin_Latch_Write(1);
    Pin_Latch_Write(0);
}

static uint8_t led_counter = 0;

CY_ISR(Timer_Int_Handler2)
{
    uint8_t dot = (led_counter == 2) ? 1 : 0;
    FourDigit74HC595_sendOneDigit(led_counter, display_data[led_counter],
dot);

    led_counter++;
    if (led_counter > 7) led_counter = 0;
}

int main(void)
{
    CyGlobalIntEnable;
    Timer_Start();
    Timer_Int_StartEx(Timer_Int_Handler2);

    uint16_t total_seconds = 300;

    for(;;)
    {
        uint8_t minutes = total_seconds / 60;
        uint8_t seconds = total_seconds % 60;

        display_data[3] = minutes / 10;
        display_data[2] = minutes % 10;
        display_data[1] = seconds / 10;
        display_data[0] = seconds % 10;

        CyDelay(1000);

        if (total_seconds > 0) {
            total_seconds--;
        } else {
            display_data[3] = 20; display_data[2] = 20;
            display_data[1] = 20; display_data[0] = 20;
            CyDelay(500);
            display_data[3] = 0; display_data[2] = 0;
            display_data[1] = 0; display_data[0] = 0;
            CyDelay(500);
        }
    }
}

```


Висновки

Під час виконання лабораторної роботи №5 було успішно досліджено принципи динамічної індикації та налаштування апаратних переривань мікроконтролерів PSoC. Експериментально встановлено, що оптимальний період таймера 1 мс забезпечує стабільне відображення даних, тоді як його збільшення спричиняє видиме мерехтіння дисплея. За допомогою логічного аналізатора Saleae Logic декодовано сигнали інтерфейсу SPI та підтверджено коректність передачі шістнадцяткових кодів на зсувні регістри 74HC595 для виведення заданого числа. У ході виконання самостійних завдань реалізовано алгоритм опитування матричної клавіатури для перевірки пароля з асинхронним візуальним сповіщенням, що не блокує розгортку екрана. Також успішно запрограмовано циклічний зсув символів ліворуч та створено 5-хвилинний зворотний відлік часу із динамічною крапкою. Загалом робота дозволила опанувати ефективне поєднання апаратних і програмних ресурсів для керування периферією в режимі реального часу.